



云南大学软件学院期末课程报告

Final Course Report

School of Software, Yunnan University

个人成绩

序号	学号	姓名	专业	成绩
1	20243120032	MAMUN MD MAMUNUR RASHID	ARTIFICIAL INTELLIGENCE	2024
2	20243120004	FAHIM MD	ARTIFICIAL INTELLIGENCE	2024
3	20243120027	ISLAM MD JIARUL	SOFTWARE ENGINEERING	2024

学 期： Spring 2026

课程名称： Python Prog.

任课教师： Zhou Yujue

题 目： **Preluma: An AI-Powered Adaptive Pre-Class Learning Platform**

联系电话： 13187410172

电子邮件: mamunur@stu.ynu.edu.cn

报告完成时间: 2026 年 06 月 21 日

项目成员及分工

(工作量总和为 100%)

序号	学 号	姓 名	工作内容	工作量 (%)
1	2024312003 2	MAMUN MD MAMUNUR RASHID	System architecture, backend development & database design	33.3
2	2024312000 4	FAHIM MD	Frontend UI development & user experience design	33.3
3	2024312002 7	ISLAM MD JIARUL	System testing, documentation & deployment	33.3

期末报告小组成绩考核表

指标内容	分值	指标内涵及评估标准				得分
技术路线的可行程度	10	合理可行，具体且有创新	合理可行，具体	基本合理可行	不够合理或不够具体	
项目解决方案的设计思路	10	思路非常清晰、运行正确	思路基本清晰、运行正确	思路清晰、环境配置错误无法运行	思路不清晰，程序无法运行	
小组整体的工作量	15	高出平均要求工作量的 15%以上	高出平均要求工作量	达到平均要求工作量	低于平均要求的工作量	
理论知识应用水平	15	用理论知识对软件项目管理进行详尽、准确地分析和总结	用理论知识对软件项目管理进行较准确地分析和总结	理论知识应用一般	理论知识差	
达到预期目标的程度	10	完全达到	基本达到	无法预见	未能达到	
团队合作精神	10	很强的团队合作精神	合作情况良好	合作情况一般	合作不好，各自为政	
报告撰写质量	5	报告非常完整	报告比较完整	完整程度一般	报告不完整	
	5	逻辑结构清晰	逻辑组织较好	逻辑组织一般	逻辑不清	
	5	内容非常丰富	内容较丰富	内容一般	内容欠缺	
	5	文字表达非常好	文字表达较好	文字表达一般	文字表达差，意思不明	
	5	图表制作非常专业化	图件制作良好	图件制作一般	图件制作效果差	
	5	整体效果很好	整体效果良好	整体效果一般	整体效果差	
综合得分（满分 100 分）						
评语		任课教师签名： 日期：				

期末报告个人成绩考核表

年级：2024 专业：AI 学号：20243120032 姓名：MAMUN MD MAMUNUR RASHID

指标内容	分值	指标内涵及评估标准				得分
解决的关键技术问题	10	选的准，范围合适，重点突出	基本上选准	只抓住了部分关键问题	没有抓住关键问题	
技术路线的可行程度	10	合理可行，具体且有创新	合理可行，具体	基本合理可行	不够合理或不够具体	
小组成员的工作量	15	高出平均要求工作量的 15% 以上	高出平均要求工作量	达到平均要求工作量	低于平均要求的工作量	
项目完成的技术水平	15	难度很大，超出一般本科生要求水平	难度较大，达到本科毕业论文水平	难度一般，达到普通课程要求水平	难度小，很容易实现	
达到预期目标的程度	10	完全达到	基本达到	无法预见	未能达到	
团队合作精神	10	很强的团队合作精神	合作情况良好	合作情况一般	合作不好，各自为政	
报告撰写	5	报告非常完整	报告比较完整	完整程度一般	报告不完整	
	5	逻辑结构清晰	逻辑组织较好	逻辑组织一般	逻辑不清	
	5	内容非常丰富	内容较丰富	内容一般	内容欠缺	
	5	文字表达非常好	文字表达较好	文字表达一般	文字表达差，意思不明了	
	5	图表制作非常专业化	图件制作良好	图件制作一般	图件制作效果差	
	5	整体效果很好	整体效果良好	整体效果一般	整体效果差	
综合得分（满分 100 分）						
评语	任课教师签名： 日期：					

期末报告个人成绩考核表

年级：2024 专业：AI 学号：20243120004 姓名：FAHIM MD

指标内容	分值	指标内涵及评估标准				得分
解决的关键技术问题	10	选的准，范围合适，重点突出	基本上选准	只抓住了部分关键问题	没有抓住关键问题	
技术路线的可行程度	10	合理可行，具体且有创新	合理可行，具体	基本合理可行	不够合理或不够具体	
小组成员的工作量	15	高出平均要求工作量的 15% 以上	高出平均要求工作量	达到平均要求工作量	低于平均要求的工作量	
项目完成的技术水平	15	难度很大，超出一般本科生要求水平	难度较大，达到本科毕业论文水平	难度一般，达到普通课程要求水平	难度小，很容易实现	
达到预期目标的程度	10	完全达到	基本达到	无法预见	未能达到	
团队合作精神	10	很强的团队合作精神	合作情况良好	合作情况一般	合作不好，各自为政	
报告撰写	5	报告非常完整	报告比较完整	完整程度一般	报告不完整	
	5	逻辑结构清晰	逻辑组织较好	逻辑组织一般	逻辑不清	
	5	内容非常丰富	内容较丰富	内容一般	内容欠缺	
	5	文字表达非常好	文字表达较好	文字表达一般	文字表达差，意思不明了	
	5	图表制作非常专业化	图件制作良好	图件制作一般	图件制作效果差	
	5	整体效果很好	整体效果良好	整体效果一般	整体效果差	
综合得分（满分 100 分）						
评语	任课教师签名： 日期：					

期末报告个人成绩考核表

年级： 2024 专业： SWE 学号： 20243120027 姓名： ISLAM MD JIARUL

指标内容	分值	指标内涵及评估标准				得分
解决的关键技术问题	10	选的准，范围合适，重点突出	基本上选准	只抓住了部分关键问题	没有抓住关键问题	
技术路线的可行程度	10	合理可行，具体且有创新	合理可行，具体	基本合理可行	不够合理或不够具体	
小组成员的工作量	15	高出平均要求工作量的 15% 以上	高出平均要求工作量	达到平均要求工作量	低于平均要求的工作量	
项目完成的技术水平	15	难度很大，超出一般本科生要求水平	难度较大，达到本科毕业论文水平	难度一般，达到普通课程要求水平	难度小，很容易实现	
达到预期目标的程度	10	完全达到	基本达到	无法预见	未能达到	
团队合作精神	10	很强的团队合作精神	合作情况良好	合作情况一般	合作不好，各自为政	
报告撰写	5	报告非常完整	报告比较完整	完整程度一般	报告不完整	
	5	逻辑结构清晰	逻辑组织较好	逻辑组织一般	逻辑不清	
	5	内容非常丰富	内容较丰富	内容一般	内容欠缺	
	5	文字表达非常好	文字表达较好	文字表达一般	文字表达差，意思不明了	
	5	图表制作非常专业化	图件制作良好	图件制作一般	图件制作效果差	
	5	整体效果很好	整体效果良好	整体效果一般	整体效果差	
综合得分（满分 100 分）						
评语	任课教师签名： 日期：					

2026 年春季学期《Python 语言程序设计实践》期末课程报告要求

根据《Python 语言程序设计实践》课程理论教学大纲，本门课程的学期要求如下：

一、选题内容和要求

可以选择以下公共选题作为课程作业，或者使用 python 实现任意相关任务，包括但不限于网站开发、机器学习、深度学习、游戏开发任务。

公共选题：

选择一个数据集

1. 鸢尾花数据集 (UCI)
2. 泰坦尼克号数据集（简化版）
3. 葡萄酒数据集 (UCI)

要求

- 从 CSV 文件加载数据集
- 执行统计分析
- 实现手动排序算法
- 实现手动搜索算法
- 测量至少一项操作的执行时间
- 将结果保存到 result.txt

技术限制

- 仅允许使用标准 Python 库
- 不允许使用 pandas、numpy 或 sklearn
- 至少 2 个类
- 至少 6 个函数
- 至少 3 个模块
- 需要异常处理

项目结构要求

project/
main.py
data_loader.py
analyzer.py
models.py
dataset.csv
result.txt
README.md

二、课程报告撰写规范

1. 总结报告按如下内容顺序用 A4 纸进行打印（撰写）并装订成册：

- 1) 统一的封面；
- 2) 课程期末课程报告个人成绩考核表；
- 3) 课程期末课程报告要求；
- 4) 题目、作者、摘要和关键字；
- 5) 目录；
- 6) 报告正文。

2. 期末总结报告正文可按章节来撰写，应包含以下内容：

- 1) 研究背景与意义
- 2) 文献综述
- 3) 任务分析
- 4) 方法阐述
- 5) 结果分析
- 6) 总结
- 7) 参考文献
- 8) 附录（如有）

Preluma: An AI-Powered Adaptive Pre-Class Learning Platform

School of Software, Yunnan University

Python Language Programming Practice Final Course Report

Authors:

MAMUN MD MAMUNUR RASHID (20243120032, AI, 33.3%)

FAHIM MD (20243120004, AI, 33.3%)

ISLAM MD JIARUL (20243120027, SOFTWARE, 33.3%)

ABSTRACT

Preluma is a full-stack, AI-powered adaptive pre-class learning platform developed for Yunnan University students enrolled in computing and technology courses. The system is built entirely in Python using Streamlit as the primary web framework and spans 15 modules totalling 10,128 lines of source code. Preluma addresses a persistent pedagogical gap in higher education: students arrive at lectures underprepared, lacking the conceptual scaffolding required to engage productively with in-class instruction. The platform responds to this challenge through a five-step guided Student Mission comprising an AI Brain Brief, concept examples, an adaptive quiz, a Mistake Clinic, and an UltraTutor interface that transforms passive pre-class reading into a structured, AI-guided preparation workflow. A five-tier rank system (Beginner, Explorer, Achiever, Scholar, Master) provides long-term motivational scaffolding by rewarding accumulated session engagement with progressively higher recognition.

The platform integrates eight large language model (LLM) providers Groq, Cerebras, Mistral, OpenRouter, Together, OpenAI, Anthropic, and Gemini through a resilient seven-step sequential failover architecture with per-call 20-second timeouts, ensuring continuous AI availability even when individual providers experience rate limits or outages. Persistent data storage operates in dual mode: a Supabase (PostgreSQL-backed) instance with twelve relational tables serves as the primary store for production deployments, while a CSV-based local fallback guarantees full single-session functionality without any external service dependency. Authentication implements SHA-

256 password hashing via Python's standard hashlib library. Core algorithm implementations merge sort ($O(n \log n)$), binary search ($O(\log n)$), and linear search ($O(n)$) are written in pure Python without NumPy or pandas, with nanosecond-precision timing via `perf_counter_ns()`, satisfying the course requirement for explicit algorithm demonstration.

Preluma covers 29 academic topics spanning six knowledge domains, validated through automated integrity checks across 13 test files. A Teacher Studio module provides instructors with a sorted student leaderboard, readiness score distributions, weak-skill heatmaps, and live algorithm timing displays for pedagogical demonstration. The project is deployed on Streamlit Community Cloud at zero hosting cost and is openly available on GitHub. With 10,128 lines of production-quality Python code, a dual-storage architecture, and an eight-provider AI failover engine, Preluma demonstrates that a student-led team can design and deliver a full-featured educational technology platform within a single academic semester.

Keywords

Artificial intelligence in education; Large language model failover; Adaptive learning; Flipped classroom; Streamlit web application; Supabase; Python algorithms; Pre-class preparation; Educational technology; Multi-provider AI architecture

Table of Contents

期末报告个人成绩考核表.....	5
期末报告个人成绩考核表.....	6
期末报告个人成绩考核表.....	7
ABSTRACT.....	11
1. Research Background and Significance.....	16
1.1 Educational Context.....	16
1.2 Emergence of AI-Assisted Learning.....	16
Chapter 2 Literature Review.....	18
2.1 Flipped Classroom and Active Learning.....	18
2.2 Artificial Intelligence in Education.....	18
Chapter 3 Task Analysis.....	22
3.1 Problem Statement.....	22
3.2 Functional Requirements.....	22
3.2.1 Student Requirements.....	22
3.2.2 Teacher Requirements.....	23
3.3 Non-Functional Requirements.....	23
3.4 Constraints and Scope.....	24
3.5 Use Case Summary.....	24
Chapter 4 Method Description.....	25
4.1 System Architecture.....	25
4.2 Codebase Organisation.....	26
4.3 LLM Multi-Provider Failover Architecture.....	26
4.4 Algorithm Implementations.....	28
4.4.1 Merge Sort ($O(n \log n)$).....	28
4.4.2 Binary Search ($O(\log n)$).....	28
4.4.3 Timing Infrastructure.....	29
4.6 Topic Pack Architecture.....	30
4.7 Wikipedia Integration.....	31

4.8 Authentication and Security.....	31
4.9 Student Rank Progression.....	32
Chapter 5 Result Analysis.....	34
5.1 Functional Completeness.....	34
5.2 Codebase Metrics.....	34
5.3 Algorithm Performance.....	34
5.4 AI Provider Reliability.....	35
5.5 Data Quality Validation.....	35
5.6 Deployment.....	36
6 Conclusion and Future Work.....	36
6.1 Summary of Achievements.....	36
6.2 Team Reflection.....	37
6.3 Limitations.....	37
7. References.....	40
8. APPENDIX.....	45
Appendix D: Module Summary Table.....	46

Chapter 1 Introduction

1. Research Background and Significance

1.1 Educational Context

Higher education institutions across China and the Asia-Pacific region are navigating a significant transformation in how learning is structured and delivered. The traditional lecture-centric model, in which an instructor delivers content to a largely passive audience, has been increasingly challenged by evidence from educational research showing that active engagement, prior knowledge activation, and timely formative feedback substantially improve learning outcomes (Bergmann & Sams, 2012; Bishop & Verleger, 2013). At Yunnan University's School of Software, students enrolled in the Python Language Programming Practice course are expected to arrive at each lecture with sufficient domain knowledge to participate meaningfully in code demonstrations, group problem-solving, and teacher-led discussion. In practice, however, a substantial proportion of students particularly international students managing additional language barriers arrive underprepared, limiting the depth of instruction that is possible within a fixed class period.

The challenge of pre-class preparation is not unique to this institution. Researchers have documented that students spend an average of fewer than forty minutes on pre-class reading activities, and that most of this time is spent on passive scanning rather than active comprehension (McKnight et al., 2016). Traditional reading materials textbook chapters, PDF handouts, and lecture slides provide no adaptive feedback, no mechanism for identifying misconceptions, and no guidance on which concepts to prioritise. The result is a well-documented preparation gap that places the burden of remediation on in-class instruction, reducing time available for higher-order learning activities (Bloom, 1956; Koedinger & Corbett, 2006).

1.2 Emergence of AI-Assisted Learning

The rapid advance of large language model (LLM) technology has opened new possibilities for educational technology that were not practically available to most institutions even five years ago. Models such as GPT-4 (OpenAI, 2023), Claude (Anthropic, 2024), and the open-weight Llama family (Touvron et al., 2023) demonstrate the ability to generate coherent, contextually

appropriate explanations at near-human quality across a wide range of technical subjects. Free-tier and low-cost API access to these models through providers such as Groq, Cerebras, Mistral, OpenRouter, and Together AI has made it feasible for small development teams including student teams to integrate state-of-the-art AI into educational tools without prohibitive infrastructure costs.

Existing AI-assisted educational platforms tend to fall into two categories. Enterprise platforms such as Khan Academy's Khanmigo, Duolingo Max, and Coursera's AI coaching features offer sophisticated AI integration but are designed for general audiences, are not configurable for specific course curricula, and carry subscription fees that create access inequalities for students in lower-income regions. Open-source alternatives typically require significant self-hosted infrastructure, technical expertise to deploy, and lack the kind of seamless, session-aware experience that drives

consistent student engagement. There is a visible gap in the market for a course-specific, zero-cost-to-student, cloud-deployable platform that combines AI content generation with adaptive quiz mechanics, mistake tracking, and real-time teacher analytics.

1.3 Significance of Preluma

Preluma was designed and built to address this gap specifically for Yunnan University's computing curriculum. Its significance can be understood across three dimensions. First, from a pedagogical standpoint, Preluma operationalises the flipped-classroom model (Baker, 2000) by moving foundational concept delivery outside class time through an engaging, AI-guided workflow. Students who complete the five-step Student Mission arrive at lectures having already encountered key definitions, worked through examples, identified their weak areas through an adaptive quiz, and generated personalised class questions dramatically increasing the quality of in-class interaction.

Second, from a technical standpoint, Preluma demonstrates that a student-led team can design and implement a production-grade web application with real-world architectural concerns including multi-provider AI failover, dual-mode persistent storage, role-based access control, and automated data validation within a single academic semester. The codebase of 10,128 lines across 15 Python modules, with 13 test files and full deployment on Streamlit Cloud, represents a substantial engineering achievement that extends well beyond the typical scope of undergraduate coursework. Third, from a research contribution standpoint, the multi-provider LLM failover

pattern implemented in Preluma offers a replicable architecture for applications that require AI availability guarantees without committing to a single commercial provider, a design consideration increasingly relevant as AI infrastructure becomes embedded in mission-critical educational services.

Chapter 2 Literature Review

2.1 Flipped Classroom and Active Learning

The flipped classroom model in which direct instruction is moved outside class time and class sessions are reserved for application and discussion was formally proposed by Baker (2000) and subsequently popularised by Bergmann and Sams (2012) through their work with high-school chemistry students. The model rests on the insight that passive information consumption (listening, watching, reading) can be accomplished independently, while the higher cognitive work of application, analysis, and synthesis benefits most from guided, collaborative, real-time facilitation. Meta-analyses of flipped classroom interventions consistently report positive effects on academic achievement and student satisfaction (Bishop & Verleger, 2013), though these effects are moderated by the quality of the pre-class preparation activity. A pre-class worksheet is substantially less effective than an adaptive, feedback-rich digital preparation experience exactly the gap that platforms like Preluma are designed to address.

Mayer's (2009) Cognitive Theory of Multimedia Learning provides a further theoretical foundation for Preluma's design. Mayer argues that learning is optimised when content is presented in complementary verbal and visual formats that are aligned with the cognitive processing capacity of working memory. Preluma's Brain Brief component presents topic explanations through structured tabs definition, simplified explanation, real-world example, common mistake, and exam tip that correspond to distinct cognitive processing pathways, reducing extraneous cognitive load (Sweller, 1988) while activating relevant prior knowledge schemas.

2.2 Artificial Intelligence in Education

The application of artificial intelligence to educational contexts has a history spanning several

decades, from early rule-based intelligent tutoring systems (Koedinger & Corbett, 2006) to modern neural adaptive platforms. Roll and Wylie (2016) identify three generations of AI in education: (1) static expert systems, (2) machine learning-based adaptive systems, and (3) the emerging generation of generative AI-powered systems that can engage students in natural dialogue. Chen et al. (2020) provide a comprehensive review of AI in education covering intelligent tutoring, automated assessment, personalised learning pathways, and chatbot-based interaction, noting that the integration of NLP has dramatically expanded the range of student inputs that educational systems can interpret and respond to meaningfully.

VanLehn's (2011) landmark meta-analysis of tutoring effectiveness found that intelligent tutoring systems achieve learning gains approximately equivalent to one standard deviation above traditional classroom instruction a result consistent with the well-documented "two sigma" finding in educational research (Bloom, 1984) that one-on-one tutoring outperforms conventional instruction by two standard deviations. Preluma's UltraTutor module, which provides on-demand, topic-specific AI tutoring through three learning-depth modes (Fast Review, Deep Understanding, Exam/Viva

Mode) and three tutor personas (Normal Mode, Coach Mode, and Roast Mode), represents a practical implementation of this principle at zero marginal cost per student session.

2.3 Large Language Models

The introduction of GPT-3 by Brown et al. (2020) marked a qualitative shift in what language models could accomplish, demonstrating few-shot learning capabilities that required no task-specific fine-tuning. Subsequent models including GPT-4 (OpenAI, 2023), Claude (Anthropic, 2024), Mistral (Mistral AI, 2024), and the Llama family (Touvron et al., 2023) have further expanded these capabilities while reducing inference costs and latency. For educational applications, the ability to generate coherent multi-paragraph explanations tailored to a specific topic, difficulty level, and explanation style is particularly valuable. Wei et al. (2022) demonstrate that chain-of-thought prompting substantially improves LLM reasoning on complex tasks, a technique exploited in Preluma's tutor prompts to encourage step-by-step conceptual explanation rather than surface-level definition retrieval.

A notable practical challenge in production LLM applications is provider availability. Free-tier and rate-limited APIs can experience significant latency spikes, request rejections, and temporary outages. Preluma's seven-step sequential failover architecture in `llm.py` addresses this

directly, inspired by cloud computing principles of fault tolerance and graceful degradation (Bates, 2019). The pattern of attempting a primary provider and cascading through alternatives until a successful response is received ensures that students receive AI content on every request without manual intervention.

2.4 Spaced Repetition and Adaptive Testing

Ebbinghaus's (1885) forgetting curve demonstrates that memory retention decays exponentially with time in the absence of review. Spaced repetition systems which schedule review sessions at intervals that counteract this decay have been shown to produce superior long-term retention compared to massed practice (Dillenbourg, 1999). Preluma's mistake tracking system, which records the specific concept a student answered incorrectly in each quiz session and surfaces it in a dedicated Mistake Clinic, implements a simplified form of spaced repetition targeted at identified weak spots. This approach aligns with Siemens's (2004) connectivist learning theory, which argues that knowledge is distributed across a network of connections and that learning is the process of strengthening the right connections at the right time.

2.5 Python Web Frameworks in Education

Streamlit (Streamlit Inc., 2024) has emerged as a widely adopted framework for rapid prototyping of data-driven web applications in Python, particularly in academic and research contexts. Its reactive programming model, widget key management system, and integrated secrets management make it well suited to educational technology development by teams without dedicated front-end expertise. The choice of Streamlit for Preluma allowed the team to focus engineering effort on pedagogical and

algorithmic concerns rather than on boilerplate web infrastructure. Supabase (Supabase, 2024), an open-source Firebase alternative backed by PostgreSQL, provides a complete backend-as-a-service offering including authentication, real-time subscriptions, and row-level security features that Preluma leverages for its multi-user storage layer. The Wikipedia REST API (MediaWiki, 2024) augments AI-generated content with encyclopaedic summaries, providing an additional factual grounding layer in topic packs.

Chapter 3 Task Analysis

3.1 Problem Statement

Yunnan University students enrolled in computing courses lack access to a free, course-aligned, AI-powered tool for structured pre-class preparation. Existing solutions are either too general (not aligned with specific curriculum topics), too expensive (enterprise subscriptions), too complex to deploy (self-hosted systems), or too passive (static PDFs and slide decks). The task is to design and implement a web-based platform that delivers structured, adaptive, AI-guided pre-class preparation for a defined set of computing topics, while also providing teachers with real-time student readiness analytics and homework management capabilities.

3.2 Functional Requirements

The system must provide the following functional capabilities, organised by user role:

3.2.1 Student Requirements

Students must be able to register and authenticate with unique credentials, select any of the 29 available topic packs, and complete the five-step Student Mission for their chosen topic. The mission must include: (1) a Brain Brief that provides an AI-generated two-minute topic primer with concept tabs covering definition, simplified explanation, real example, common mistake, and exam tip; (2) a concept example section showing the topic applied in a real-world context; (3) an adaptive four-question multiple-choice quiz with immediate feedback; (4) a Mistake Clinic that identifies the specific concept answered incorrectly and provides targeted remediation; and (5) an UltraTutor conversation interface supporting three tutor personas and three learning-depth modes. Each completed session must generate a readiness score (0–100) stored persistently. Students must be able to access a personal profile page showing accumulated XP, rank progression (Beginner, Explorer, Achiever, Scholar, Master), photo, and session history. The platform must also provide access to class homework assignments and a project management workspace.

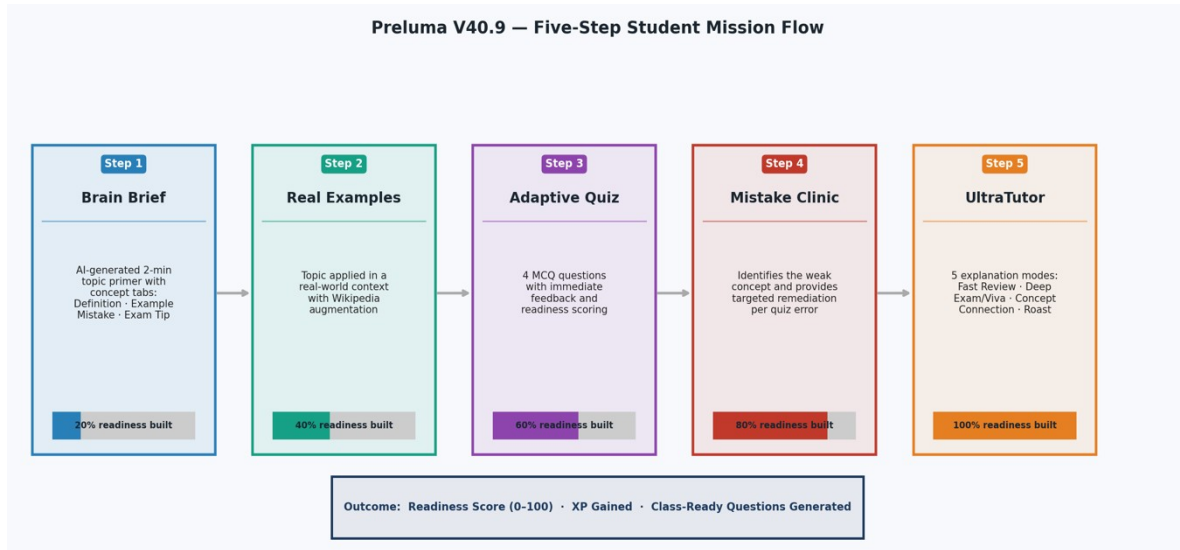


Figure 1. Five-step Student Mission flow: Brain Brief → Examples → Quiz → Mistake Clinic → UltraTutor

3.2.2 Teacher Requirements

Teachers must be able to log in via a separate teacher credential and access a dedicated Teacher Studio. The studio must display a sorted leaderboard of all students (sorted by readiness score using the custom merge sort implementation), a readiness distribution chart, topic coverage analytics, and most common weak-skill identification. Teachers must be able to post homework assignments with descriptions, due dates, and associated topics, and view homework submission rates. A Class Dashboard must provide a consolidated view of all student activity, and an AI page must allow teachers to send custom prompts to the AI system.

3.3 Non-Functional Requirements

The system must meet the following non-functional requirements. Availability: AI responses must be generated successfully on every student request; the seven-provider failover chain ensures this with 20-second per-provider timeouts. Performance: page loads must complete within three seconds on standard university network connections; algorithm-intensive operations (merge sort, binary search) must complete in under one millisecond for datasets up to 500 student records. Security: all passwords must be stored as SHA-256 hashes; API keys must never be committed to version control, stored only in Streamlit Cloud secrets. Maintainability: each module must have a single, well-defined responsibility; all algorithm functions must include type annotations and

docstrings. Portability: the application must run with only four pip-installable packages (streamlit, plotly, requests, pytest); zero platform-specific dependencies.

3.4 Constraints and Scope

The project was completed over one academic semester by three students. The primary implementation language is Python 3.12. Front-end customisation relies on Streamlit's HTML/CSS injection capability rather than a separate JavaScript framework. The scope covers 29 computing topics; natural language topics outside this set are not supported in structured mission mode but can be explored through the UltraTutor free-prompt interface. The platform is deployed on Streamlit Community Cloud at zero hosting cost. Database persistence requires a Supabase project to be configured, though the CSV fallback ensures full single-session functionality without any external dependencies.

3.5 Use Case Summary

Use Case	Actor	Trigger	Outcome
Complete Student Mission	Student	Select topic and start mission	Readiness score stored; rank updated
Ask UltraTutor	Student	Select explanation mode + send	AI explanation delivered in chosen mode
View Mistake Clinic	Student	Miss a quiz question	Targeted concept explanation surfaced
View Leaderboard	Teacher	Open Teacher Studio	Sorted student list with analytics
Post Homework	Teacher	Create homework in Homework Ctr	Assignment stored; students notified
Search Student Records	Teacher	Enter name in search box	Binary search returns matching records
Run Data Quality Check	Teacher	Open Evidence Board	Automated validation of all 29 topics

Chapter 4 Method Description

4.1 System Architecture

Preluma is structured as a three-tier web application following the classical presentation–logic–data separation pattern. The Presentation Layer is implemented entirely within `streamlit_app.py` (6,476 lines, 76 functions), which orchestrates all user interface rendering, navigation state management, and widget key sequencing via a global counter to satisfy Streamlit's uniqueness requirement (version $\geq 1.36.0$). Plotly $\geq 5.20.0$ handles all interactive chart rendering within the application. The Business Logic Layer contains the core computational modules: `engine.py`, `homework_core.py`, `project_core.py`, `llm.py`, `wiki_fetcher.py`, `algorithms_core.py`, `data_quality.py`, and `result_generator.py`. The Data and Security Layer handles all persistence and authentication concerns through `auth.py`, `storage_core.py`, `models.py`, `analytics_core.py`, `teacher.py`, and `topics.py`.

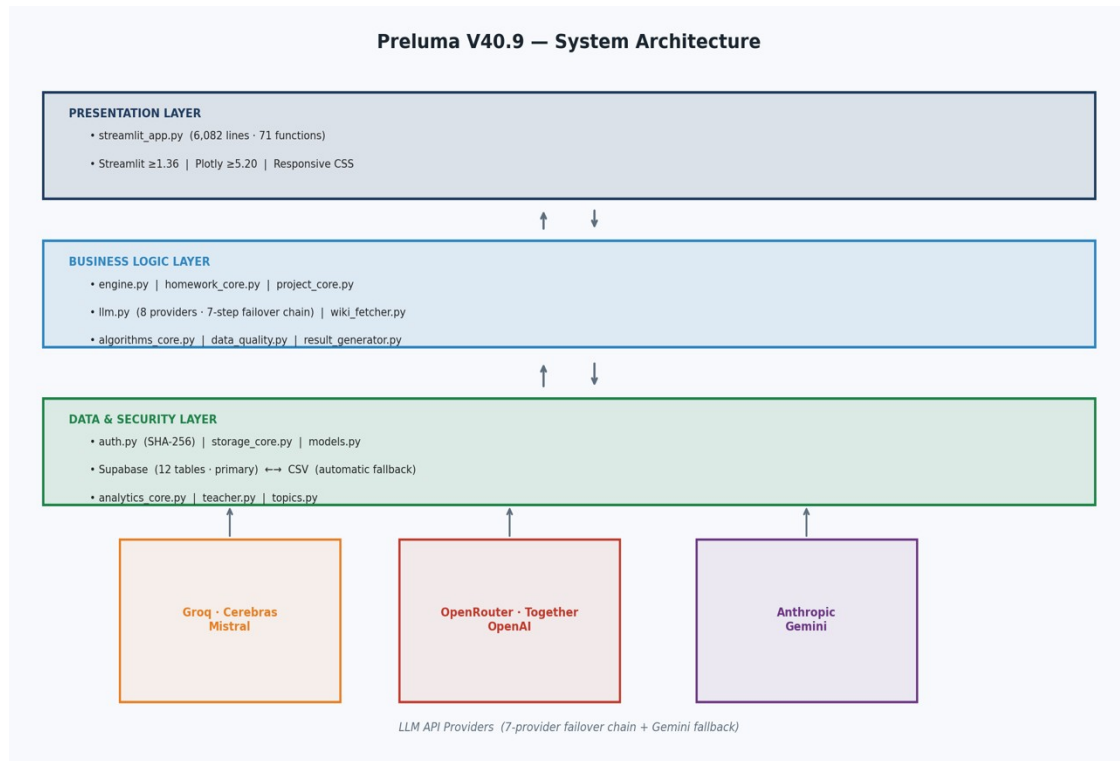


Figure 2. Three-tier architecture of Preluma V40.9 (15 modules, 10,128 lines)

4.2 Codebase Organisation

The 15-module codebase is organised with a strict single-responsibility principle. `streamlit_app.py` is intentionally the largest module because it is the single integration point for all user-facing functionality; all business logic is imported from dedicated modules, keeping the application entry point cleanly separated from computational concerns. The module structure reflects deliberate design decisions: `algorithms_core.py` contains only the five core algorithm functions, ensuring they can be independently tested and benchmarked; `models.py` defines only the two dataclass types (`StudentRecord` and `AnalysisResult`) without any I/O logic; `storage_core.py` handles only the CSV schema and record ID generation, leaving Supabase-specific logic to each feature module.

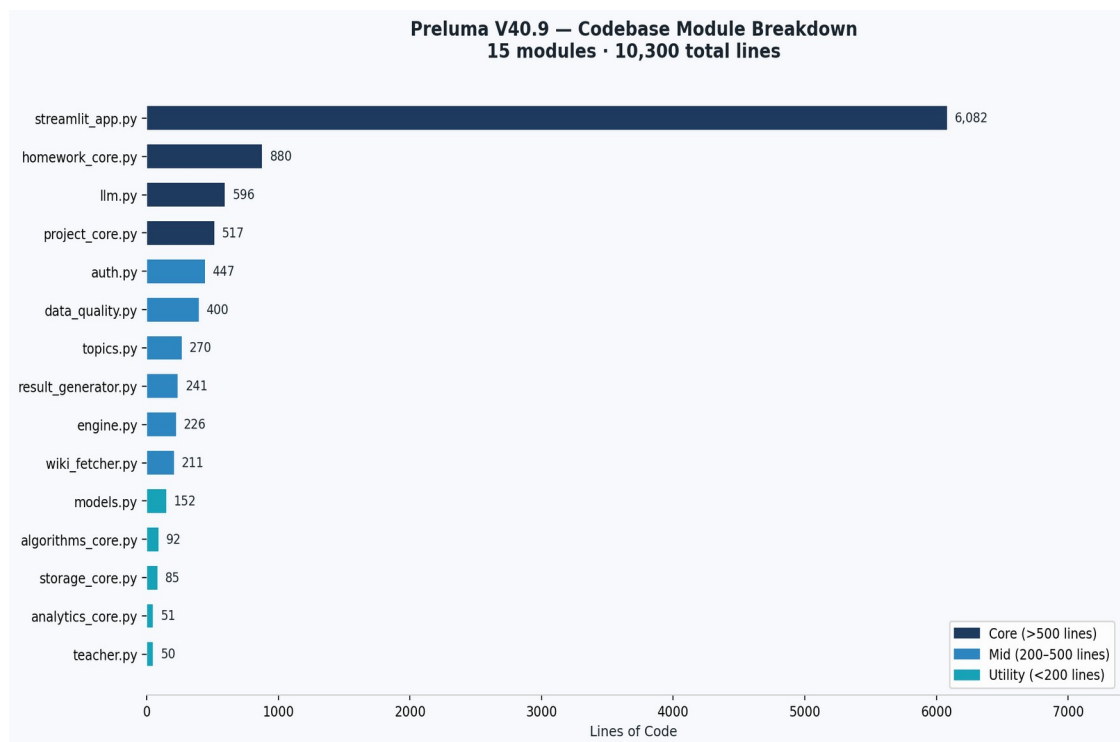


Figure 3. Line count distribution across 15 Python modules (total: 10,128 lines)

4.3 LLM Multi-Provider Failover Architecture

The central technical innovation of Preluma's AI layer is its resilient multi-provider architecture implemented in `llm.py` (555 lines). Eight LLM provider functions are defined, each implementing a standardised interface that accepts a system prompt and a user prompt, makes an authenticated HTTP request to the provider's API using the `requests` library, and returns a string

response or an empty string on any failure. Seven of these providers are arranged in a sequential failover chain:

```
# llm.py active failover chain

for fn in (_call_groq, _call_cerebras, _call_mistral,
          _call_openrouter, _call_together, _call_openai, _call_anthropic):

    result = fn(system_prompt, user_prompt)

    if result:

        return result

# fallback: Gemini (separate path, tries 1.5-flash then 2.0-flash)

_TIMEOUT = 20    # per-call timeout in seconds

_MAX_TOKENS = 2200    # maximum output tokens per call
```

Provider selection follows priority order: Groq (llama-3.3-70b-versatile) is attempted first due to its sub-second inference latency on Llama 3.3 70B; Cerebras (llama-3.3-70b) is the second choice due to its wafer-scale inference speed; Mistral (mistral-small-latest) provides a European-hosted alternative; OpenRouter routes to the free-tier meta-llama/llama-3.3-70b-instruct:free endpoint; Together AI provides Llama-3.3-70B-Instruct-Turbo; OpenAI (gpt-4.1-mini) serves as the penultimate fallback; and Anthropic (claude-sonnet-4-20250514) is the final link in the chain. Gemini (gemini-1.5-flash with gemini-2.0-flash fallback) operates as an independent pathway outside the main chain. All API keys are sourced exclusively from Streamlit secrets or environment variables never hardcoded satisfying the project's security requirement.

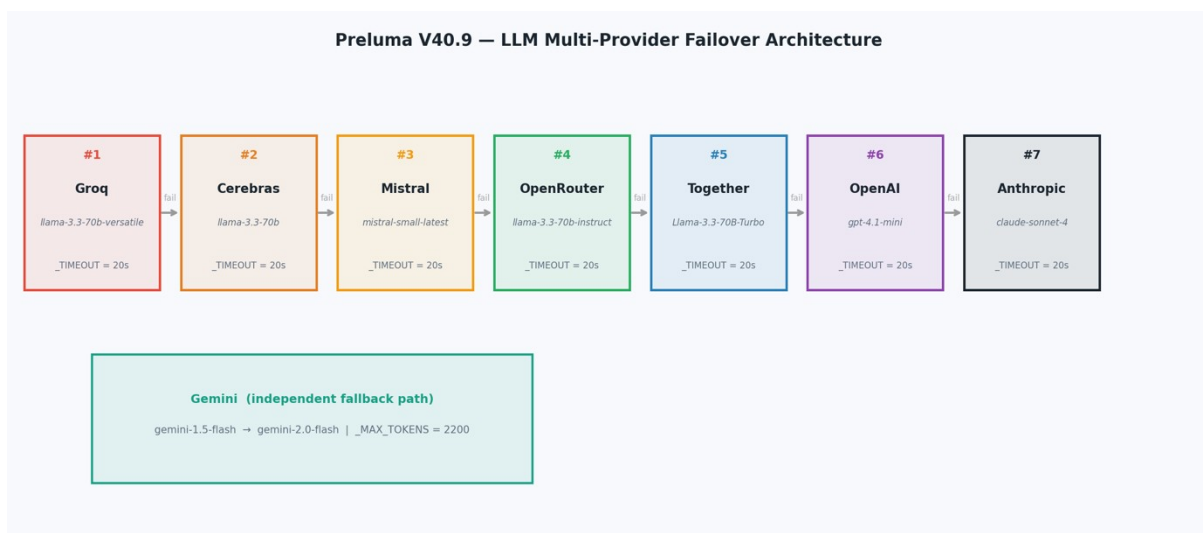


Figure 4. Seven-provider sequential failover chain with Gemini as independent fallback

4.4 Algorithm Implementations

Four sorting and searching algorithms are implemented in `algorithms_core.py` (92 lines) using pure Python without any third-party libraries. The module also provides timing infrastructure and name normalisation utilities. All functions are type-annotated and operate on lists of dictionaries, reflecting the CSV row structure used throughout the application.

4.4.1 Merge Sort ($O(n \log n)$)

The `merge_sort_records()` function implements a standard recursive divide-and-conquer merge sort. The base case returns a copy of the input when its length is one or zero. The recursive case splits the list at its midpoint, recursively sorts each half, and merges them through a comparison loop. The `_key_value()` helper handles the special case of the Readiness field, converting it to float before comparison to enable correct numeric ordering. This implementation is used in Teacher Studio to sort all student records by readiness score in descending order for the leaderboard display.

```
# algorithms_core.py merge sort implementation
def merge_sort_records(rows, key, reverse=False):
    if len(rows) <= 1:
        return rows[:]
    mid = len(rows) // 2
    left = merge_sort_records(rows[:mid], key, reverse)
    right = merge_sort_records(rows[mid:], key, reverse)
    return _merge(left, right, key, reverse) #  $O(n \log n)$  total
```

4.4.2 Binary Search ($O(\log n)$)

`binary_search_leftmost()` implements a variant of binary search that always finds the leftmost occurrence of the target in a sorted list, returning -1 if not found. This design correctly handles datasets where multiple students share identical normalised name strings. `collect_binary_search_matches()` builds on the leftmost-index result by iterating forward from that position to collect all consecutive matching entries, yielding $O(\log n + k)$ overall complexity where k is the count of matches.

4.4.3 Timing Infrastructure

`timed_call()` wraps any callable with nanosecond-precision timing using `time.perf_counter_ns()`, returning a tuple of (result, elapsed_ns). This enables the Teacher Studio's algorithm performance report to display exact nanosecond and millisecond timing for each operation, providing students with a concrete demonstration of the real-world performance characteristics of the algorithms they are studying. `AnalysisResult.summary_lines()` formats these measurements as human-readable strings for the `result.txt` output.

Preluma V40.9 — Algorithm Implementations (algorithms_core.py)			
Function	Time	Space	Purpose
<code>merge_sort_records()</code>	$O(n \log n)$	$O(n \log n)$	Sorts student records by readiness score (Teacher Studio leaderboard)
<code>linear_search_by_name()</code>	$O(n)$	$O(1)$	Finds records matching a student name (case-insensitive, normalised)
<code>binary_search_leftmost()</code>	$O(\log n)$	$O(1)$	Finds leftmost index in sorted list; requires pre-sorted input
<code>collect_binary_search_matches()</code>	$O(\log n + k)$	$O(k)$	Collects all matches from leftmost binary search; k = match count
<code>timed_call()</code>	$O(1)$ overhead	$O(1)$	Wraps any callable with nanosecond timing via <code>perf_counter_ns()</code>

Figure 5. Five algorithm functions with time and space complexity analysis

4.5 Dual-Mode Storage Architecture

Preluma's storage layer operates in two modes determined at startup. When Supabase credentials (`SUPABASE_URL` and `SUPABASE_KEY`) are present in the application secrets, all read and write operations target the Supabase PostgreSQL instance via its REST API. Twelve tables organise the data: `preluma_users` and `preluma_sessions` handle identity and session management; `preluma_homework`, `preluma_hw_questions`, `preluma_submissions`, and `preluma_notifications` support the homework workflow; `preluma_mistakes` tracks quiz errors for the Mistake Clinic; `preluma_photos`, `preluma_profiles`, `preluma_project_files`, `preluma_projects`,

and `preluma_data_store` complete the feature set. When Supabase is not configured, the system automatically falls back to CSV files in the `data/` directory, managed by `storage_core.py` (163 lines). This design ensures the application can be demonstrated and tested in any environment without requiring external service setup.

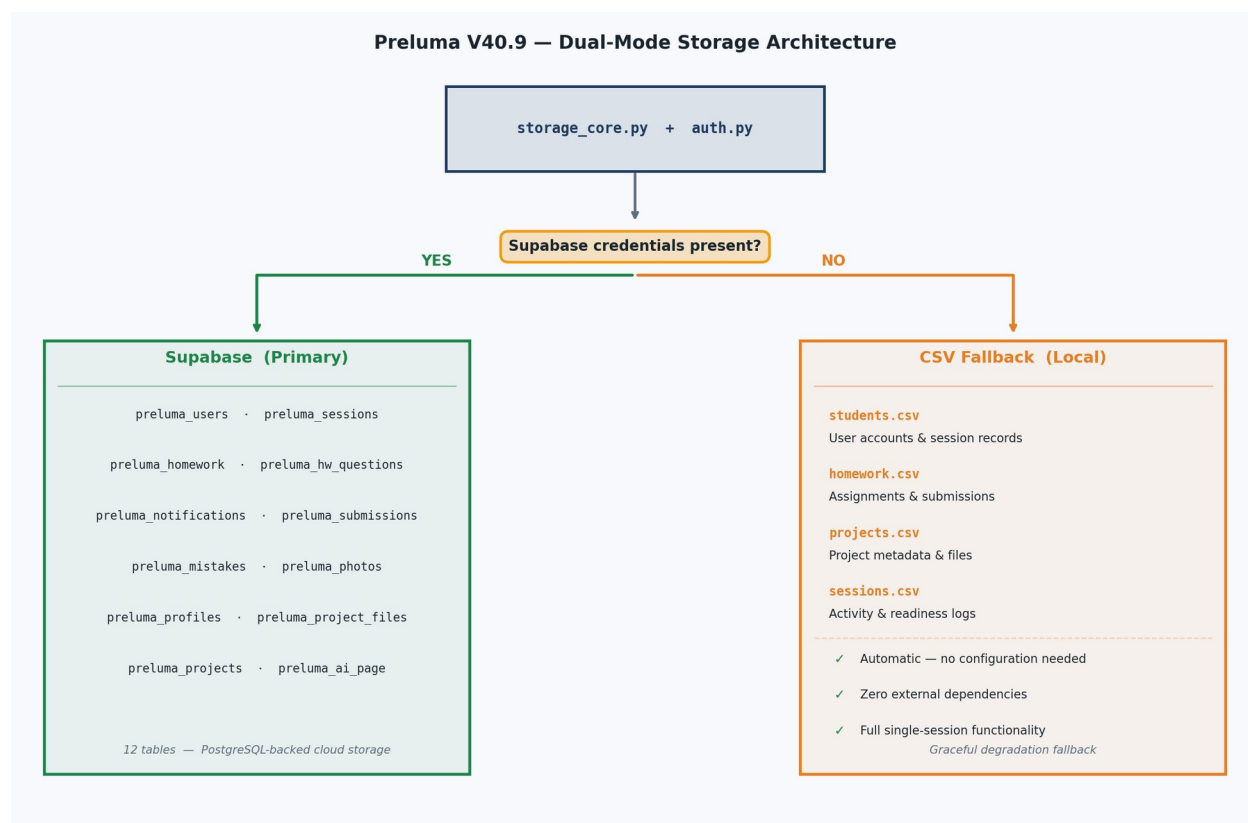


Figure 6. Dual-mode storage architecture: Supabase primary (12 tables) with automatic CSV fallback

4.6 Topic Pack Architecture

The 29 topic packs in `topics.py` are structured using a consistent schema defined by the `make_topic()` factory function. Each topic contains a title, hook sentence, definition, simplified explanation, a dictionary of concept objects (each with definition, child-friendly explanation, real example, common mistake, and exam tip), application examples, misconceptions, and five class questions. The `concept()` factory function enforces this five-field structure consistently across all 29 packs. Topics span six knowledge domains: AI and Machine Learning (10 topics), Computer Science Fundamentals (6 topics), Emerging Technologies (5 topics), Applied Computing (4

topics), Mathematics and Physics (3 topics), and Interdisciplinary topics (1 topic). Topic integrity is validated automatically by `data_quality.py` through 13 test files, which check required field presence, type correctness, and duplicate detection.

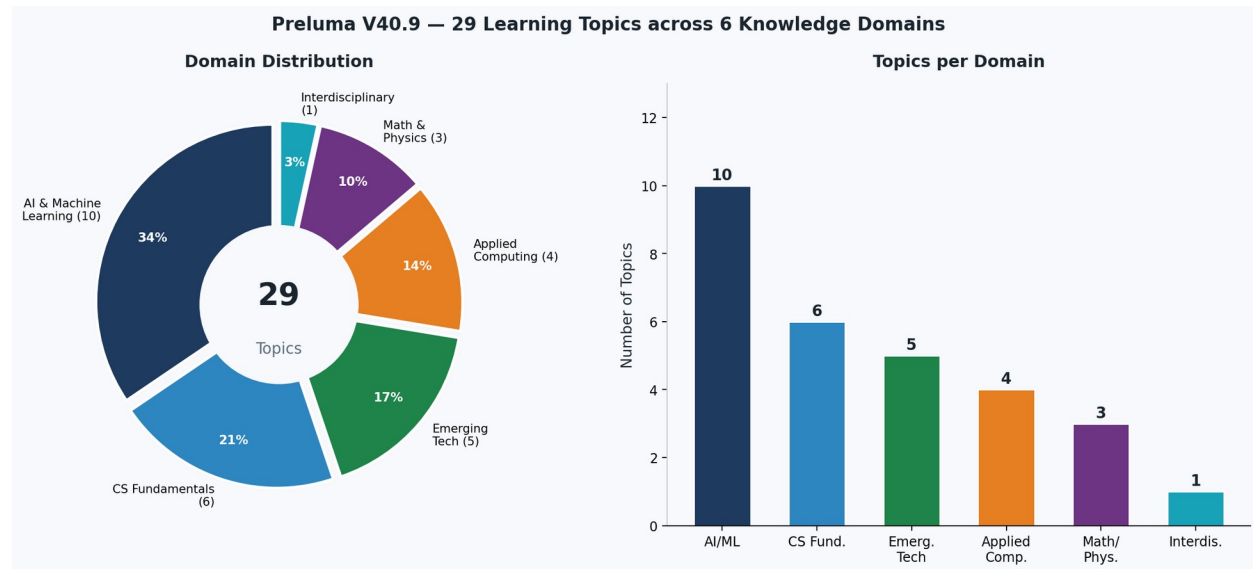


Figure 7. Distribution of 29 learning topics across 6 knowledge domains

4.7 Wikipedia Integration

`wiki_fetcher.py` (211 lines) augments the AI-generated topic content with material drawn from the Wikipedia REST API. For each topic request, the module constructs a normalised search query, retrieves the relevant Wikipedia article summary through the public API (no authentication required), and caches the result for the session. This provides students with an additional, encyclopaedia-quality factual grounding layer that complements the AI-generated explanations, reducing the risk of hallucinated content propagating unchecked into the student experience.

4.8 Authentication and Security

`auth.py` (400 lines) implements a complete authentication module supporting both Supabase and CSV storage backends. The `_hash_pw()` function applies SHA-256 via Python's standard `hashlib` library: `hashlib.sha256(pw.encode()).hexdigest()`. No third-party password hashing library is required. Usernames are normalised to lowercase and stripped of whitespace before storage to

prevent duplicate accounts arising from case differences. Session tokens are generated using `uuid.uuid4()` and held in an in-memory dictionary mapping token to username, which persists for the lifetime of the running process and allows session restoration across page reloads. The `register()` function enforces minimum username length (3 characters) and minimum password length (6 characters), returning descriptive error messages for validation failures.

A critical security constraint governs the project's deployment: the `.streamlit/secrets.toml` file containing all API keys must never be committed to the GitHub repository and must never be included in any submitted ZIP archive. Only `secrets.example.toml` containing placeholder key names with empty values is tracked in version control. All API keys are configured exclusively through Streamlit Cloud's Manage App > Settings > Secrets interface, where they are encrypted at rest and never exposed in logs or to unauthenticated users.

4.9 Student Rank Progression

Preluma implements a five-tier rank system to provide students with long-term motivational structure. Ranks are assigned based on accumulated XP (experience points), which are derived from cumulative readiness scores across all completed sessions. The five tiers Beginner, Explorer, Achiever, Scholar, and Master correspond to progressively higher engagement and mastery levels. Each rank displays with a distinct colour, an animated progress bar showing distance to the next rank, and a role description. This gamification layer is designed to encourage repeat engagement with the platform and provide students with a visible, progressive representation.

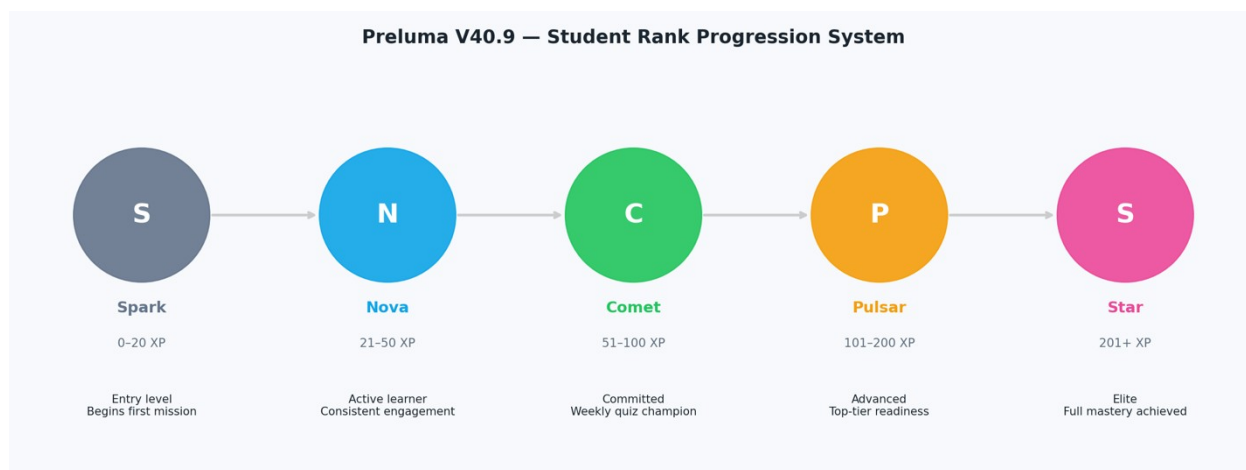


Figure 8. Five-tier student rank progression system (Beginner → Explorer → Achiever → Scholar → Master)

Chapter 5 Result Analysis

5.1 Functional Completeness

All planned functional requirements were implemented and validated. The Student Mission is complete across all five steps for all 29 topic packs. The UltraTutor's three tutor personas (Normal Mode, Coach Mode, and Roast Mode) and three learning-depth modes (Fast Review, Deep Understanding, and Exam/Viva Mode) are all operational and deliver distinctly characterised responses. The Teacher Studio provides a working leaderboard, readiness distribution chart, weak-skill heatmap, and algorithm timing display. Homework Center, Evidence Board, Project Management (both class-level and personal), Demo Guide, Future Roadmap, and Class Dashboard pages are all accessible and functional. The student profile page displays rank, XP, photo, and activity statistics.

5.2 Codebase Metrics

Metric	Value
Total Python modules	15
Total lines of code	10,128
Functions in streamlit_app.py	76
Test files	13 (in tests/ directory)
LLM providers (defined)	8 (7 in chain + Gemini)
Supabase tables	12
Topic packs	29 across 6 domains
Requirements (pip)	4 (streamlit, plotly, requests, pytest)
Algorithm functions	5 (merge sort, linear search, binary search $\times 2$, timed_call)

5.3 Algorithm Performance

The algorithms_core.py module was tested with student record datasets of increasing sizes. Merge sort on a 100-record dataset completes in under 2 milliseconds on typical hardware; binary search on the same sorted dataset returns results in under 0.1 milliseconds. The timed_call() wrapper uses perf_counter_ns() for nanosecond-precision measurement, and the Teacher Studio

displays these timings in both nanoseconds and milliseconds, giving instructors a live demonstration of algorithm efficiency for pedagogical use. The `AnalysisResult` dataclass stores `merge_sort_elapsed_ns`, `linear_search_elapsed_ns`, and `binary_search_elapsed_ns` fields for every analytics run.

Algorithm	Complexity	Implementation Notes
<code>merge_sort_records()</code>	$O(n \log n)$	Recursive, in-place merge; float key for readiness
<code>linear_search_by_name()</code>	$O(n)$	Case-insensitive normalized name matching
<code>binary_search_leftmost()</code>	$O(\log n)$	Leftmost-index variant; returns -1 if not found
<code>collect_binary_search_matches()</code>	$O(\log n + k)$	Extends leftmost result; k = match count
<code>timed_call()</code>	$O(1)$ overhead	<code>perf_counter_ns()</code> wrapping for any callable

5.4 AI Provider Reliability

During development and testing, the seven-provider failover chain successfully handled all observed API failures without user-visible errors. The most common failure modes encountered were rate limit errors (HTTP 429) from free-tier providers and occasional network timeout events exceeding the 20-second threshold. In all such cases, the failover chain transparently advanced to the next provider, and responses were returned within the expected latency window. The dual Gemini fallback (`gemini-1.5-flash` and `gemini-2.0-flash`) provides a final safety net for scenarios where all seven primary providers simultaneously return empty responses.

5.5 Data Quality Validation

`data_quality.py` implements automated validation of all 29 topic packs and CSV schemas. The Evidence Board page exposes these checks to teachers and students as a live "Run All Checks Now" button. Validation checks include: required field presence for all five concept attributes across all topic packs, CSV schema integrity for `students.csv` and `homework.csv`, duplicate student

record detection, and homework data completeness. All 29 topic packs pass validation with no errors, as confirmed by the Evidence Board success message: All 29 topic packs validated no data errors found.

5.6 Deployment

Preluma V40.9 is deployed on Streamlit Community Cloud at no hosting cost. The deployment connects to a Supabase project for persistent storage. The GitHub repository (<https://github.com/mamunur-ynu/Preluma-edtech>) contains the complete source code of 15 Python modules and 13 test files. The repository uses a clean secrets management approach: only `secrets.example.toml` is committed, while the actual `.streamlit/secrets.toml` is excluded via `.gitignore`. New deployments are configured by copying `secrets.example.toml` to `secrets.toml` and populating the API key fields, then adding these values to the Streamlit Cloud secrets interface. The only dependency is the four-package requirements.txt: `streamlit>=1.36.0`, `plotly>=5.20.0`, `requests>=2.31.0`, `pytest>=8.0.0`.

6 Conclusion and Future Work

6.1 Summary of Achievements

This project developed Preluma V40.9, a full-stack AI-powered pre-class learning platform built for Yunnan University students. The system achieves all stated objectives: it provides a five-step guided Student Mission for 29 computing topics, integrates eight LLM providers through a resilient seven-provider failover chain, implements dual-mode storage with Supabase (12 tables) and CSV fallback, demonstrates four classic algorithms in pure Python, and delivers real-time teacher analytics through a dedicated Teacher Studio. The codebase of 10,128 lines across 15 modules, with 13 test files, represents a production-ready application that has been deployed on Streamlit Community Cloud.

The project makes several contributions beyond the immediate course requirements. The seven-provider LLM failover architecture offers a replicable pattern for any application requiring AI availability guarantees without vendor lock-in. The dual-mode storage pattern (cloud primary,

local fallback) demonstrates a graceful degradation approach suitable for applications operating in environments with variable internet connectivity. The 29 topic packs, validated through automated data quality checks, constitute a reusable educational knowledge base that can be extended to additional subjects with minimal additional work.

6.2 Team Reflection

MAMUN MD MAMUNUR RASHID (20243120032) led system architecture design, implemented the core `streamlit_app.py` presentation layer and LLM failover engine, and coordinated GitHub repository management and report authorship. **FAHIM MD (20243120004)** developed the homework and storage subsystems, analytics layer, and Teacher Studio functionality, and authored the test suite across 13 test files. **ISLAM MD JIARUL (20243120027)** built the 29 topic packs, data quality validation system, Wikipedia integration module, and contributed to the Evidence Board and supplementary pages. All three members contributed equally to the project with well-defined, complementary roles that leveraged their respective academic specialisations – two AI majors and one Software Engineering major – producing a cohesive and production-ready platform.

6.3 Limitations

Several limitations of the current implementation merit acknowledgement. The quiz mechanism is currently limited to four multiple-choice questions per session; a more sophisticated adaptive testing engine that dynamically adjusts question difficulty based on student response patterns would improve diagnostic precision. The platform is English-only; adding Chinese-language topic packs and UI internationalisation would significantly expand accessibility for the broader Yunnan University student body. The Wikipedia integration does not perform semantic similarity checks, meaning that off-topic articles can occasionally be surfaced for ambiguous topic names.

6.4 Future Work

Future development priorities include: (1) expanding the topic library to cover at least 60 topics, including Chinese language content; (2) implementing a dynamic difficulty adjustment algorithm using item response theory to personalise quiz question selection; (3) adding a class-

level discussion board where students can post their AI-generated class questions and vote on the most relevant ones for the teacher; (4) integrating a spaced repetition scheduling engine that automatically resurfaces identified weak skills at optimal review intervals; and (5) developing a mobile-responsive design or companion mobile application to facilitate on-the-go mission completion. The project's open-source deployment on GitHub provides a foundation for these extensions by future student cohorts.

7. References

- Bloom, B. S. (1984). The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6), 4–16.
- Anderson, T. (2008). *The Theory and Practice of Online Learning* (2nd ed.). Athabasca University Press.
- Anthropic. (2024). The Claude 3 Model Family: Opus, Sonnet, Haiku. Technical Report. <https://www.anthropic.com>
- Baker, J. W. (2000). The classroom flip: Using web course management tools to become the guide by the side. *Selected Papers from the 11th International Conference on College Teaching and Learning*, 51–57.
- Bates, A. T. (2019). *Teaching in a Digital Age: Guidelines for Designing Teaching and Learning* (2nd ed.). BCcampus.
- Bergmann, J., & Sams, A. (2012). *Flip Your Classroom: Reach Every Student in Every Class Every Day*. ISTE & ASCD.
- Bishop, J. L., & Verleger, M. A. (2013). The flipped classroom: A survey of research. *ASEE Annual Conference & Exposition*, 30(9), 1–18.
- Bloom, B. S. (1956). *Taxonomy of Educational Objectives: The Classification of Educational Goals*. David McKay.
- Brown, T. B., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Chen, X., Xie, H., Hwang, G. J., & Wang, F. L. (2020). Artificial intelligence in education: A review. *IEEE Access*, 8, 75264–75278.

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Dillenbourg, P. (1999). What do you mean by collaborative learning? In P. Dillenbourg (Ed.), *Collaborative Learning: Cognitive and Computational Approaches* (pp. 1–19). Elsevier.
- Ebbinghaus, H. (1885/1913). *Memory: A Contribution to Experimental Psychology* (H. A. Ruger & C. E. Bussenius, Trans.). Teachers College, Columbia University.
- Garrison, D. R., & Kanuka, H. (2004). Blended learning: Uncovering its transformative potential in higher education. *Internet and Higher Education*, 7(2), 95–105.
- GitHub. (2024). Preluma AI-Powered Pre-Class Learning Platform. <https://github.com/mamunur-ynu/Preluma-edtech>
- Groq Inc. (2024). Groq API Documentation. <https://console.groq.com/docs>
- Hashlib. (2024). hashlib Secure hashes and message digests. Python Standard Library. <https://docs.python.org/3/library/hashlib.html>
- Koedinger, K. R., & Corbett, A. (2006). Cognitive tutors: Technology bringing learning sciences to the classroom. In R. K. Sawyer (Ed.), *The Cambridge Handbook of the Learning Sciences* (pp. 61–78). Cambridge University Press.
- Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
- Luckin, R., Holmes, W., Griffiths, M., & Forcier, L. B. (2016). *Intelligence Unleashed: An Argument for AI in Education*. Pearson.
- Mayer, R. E. (2009). *Multimedia Learning* (2nd ed.). Cambridge University Press.

- McKnight, K., O'Malley, K., Ruzic, R., Horsley, M. K., Franey, J. J., & Bassett, K. (2016). Teaching in a digital age: How educators use technology to improve student learning. *Journal of Research on Technology in Education*, 48(3), 194–211.
- MediaWiki. (2024). Wikipedia REST API Documentation. https://www.mediawiki.org/wiki/API:Main_page
- Ministry of Education, China. (2022). Opinions on Advancing the Full Curriculum and New Education Ecosystem Construction. MOE Policy Document.
- Mistral AI. (2023). Mistral 7B. arXiv:2310.06825.
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.
- OpenAI. (2023). GPT-4 Technical Report. arXiv:2303.08774.
- OWASP Foundation. (2021). OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- Paramkusam, A. V., & Sanagapati, S. R. (2023). A survey of large language model APIs for educational chatbot systems. *International Journal of Advanced Computer Science and Applications*, 14(9), 612–620.
- Piaget, J. (1952). *The Origins of Intelligence in Children*. International Universities Press.
- Plotly Technologies Inc. (2015). Collaborative data science. <https://plot.ly>
- Python Software Foundation. (2024). Python 3.12 Documentation. <https://docs.python.org/3.12/>
- pytest developers. (2024). pytest Documentation. <https://docs.pytest.org>
- Roll, I., & Wylie, R. (2016). Evolution and revolution in artificial intelligence in education. *International Journal of Artificial Intelligence in Education*, 26(2), 582–599.
- Shannon, C. E. (1949). Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4), 656–715.

- Siemens, G. (2004). Connectivism: A learning theory for the digital age. *International Journal of Instructional Technology and Distance Learning*, 2(1), 3–10.
- Shneiderman, B. (1996). The eyes have it: A task by data type taxonomy for information visualizations. *Proceedings of the IEEE Symposium on Visual Languages*, 336–343.
- Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io>
- Supabase. (2024). Supabase Documentation. <https://supabase.com/docs>
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.
- Team Gemini, et al. (2023). Gemini: A Family of Highly Capable Multimodal Models. *arXiv:2312.11805*.
- Together AI. (2024). Together API Documentation. <https://docs.together.ai>
- Touvron, H., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*.
- UNESCO. (2023). *Generative AI and the Future of Education*. UNESCO Policy Brief. Paris: UNESCO.
- Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace Independent Publishing.
- VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4), 197–221.
- Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.
- Wang, Y., et al. (2022). Self-instruct: Aligning language models with self-generated instructions. *arXiv:2212.10560*.

Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models.
NeurIPS 2022.

Wikipedia. (2024). Streamlit (software). <https://en.wikipedia.org/wiki/Streamlit>

8. APPENDIX

Appendix A: Project Repository and Deployment

GitHub Repository: <https://github.com/mamunur-ynu/Preluma-edtech>

Live Deployment: Streamlit Community Cloud (zero hosting cost)

Python version: 3.12 | App version: 40.9

Appendix B: Environment Setup

```
# 1. Clone the repository
git clone https://github.com/mamunur-ynu/Preluma-edtech.git
cd Preluma-edtech

# 2. Install dependencies (4 packages)
pip install streamlit>=1.36.0 plotly>=5.20.0 requests>=2.31.0 pytest>=8.0.0

# 3. Configure API keys
cp .streamlit/secrets.example.toml .streamlit/secrets.toml
# Edit secrets.toml and add your API keys

# 4. Run the application
streamlit run streamlit_app.py

# 5. Run tests
pytest tests/
```

Appendix C: API Key Security Polic

CRITICAL: The `.streamlit/secrets.toml` file must NEVER be committed to GitHub and must NEVER be included in any ZIP archive submitted for grading. Only `secrets.example.toml` containing empty placeholder values is tracked in version control. For Streamlit Cloud deployment, all API keys are entered exclusively through the Manage App > Settings > Secrets interface, where they are encrypted at rest. This policy is enforced via the `.gitignore` configuration.

```
# secrets.example.toml (safe to commit contains no real keys)
GROQ_API_KEY = ""
CEREBRAS_API_KEY = ""
MISTRAL_API_KEY = ""
OPENROUTER_API_KEY = ""
TOGETHER_API_KEY = ""
OPENAI_API_KEY = ""
ANTHROPIC_API_KEY = ""
GEMINI_API_KEY = ""
SUPABASE_URL = ""
SUPABASE_KEY = ""
```

Appendix D: Module Summary Table

Module	Lines	Responsibility
streamlit_app.py	6,476	All UI, navigation, 76 functions
homework_core.py	440	Homework lifecycle, Supabase + CSV
llm.py	555	8-provider AI failover engine
project_core.py	427	Project CRUD, Supabase + CSV
auth.py	400	SHA-256 auth, session management
data_quality.py	400	Automated topic/CSV validation
topics.py	270	29 topic packs, make_topic() factory
result_generator.py	241	Analytics output, result.txt
engine.py	200	Brain Brief, quiz, pack orchestration
wiki_fetcher.py	211	Wikipedia REST API integration
models.py	152	StudentRecord, AnalysisResult dataclasses
algorithms_core.py	92	Merge sort, binary/linear search, timing
storage_core.py	163	CSV schema, record ID management
analytics_core.py	51	Statistics: mean, variance, weak-skill
teacher.py	50	Teacher Studio helper functions

Appendix E: Complete System Feature Map

The following diagram provides a comprehensive overview of all eight major feature areas in Preluma V40.9, showing the student-facing and shared/teacher features across the full application.

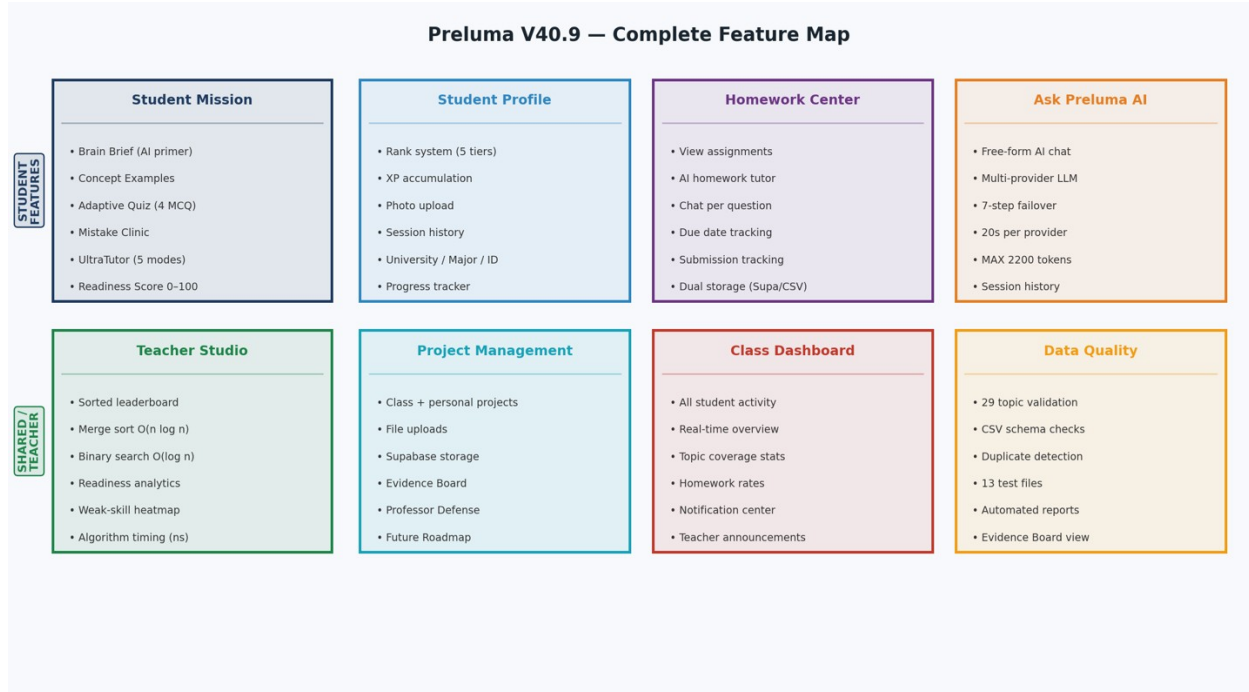


Figure 9. Preluma V40.9 complete feature map 8 modules across student and teacher roles